

# KOLEKSİYONLAR

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle karşılaştık. Programlamada en çok kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Diziler **koleksiyonlardan** (**collection**) biridir ve daha önce anlatılmıştır. Koleksiyonlar, dinamik veri yapıları olup **new** anahtar kelimesiyle çalıştırma anında bellekte oluşturulan nesnelerdir.

## ArrayList

Dizilerin çalıştırma anında boyutu belirlenir ve daha sonra boyutu değiştirilmez. Gerektiğinde boyutu dinamik olarak artırılan bir dizi kullanmak istediğimizde **ArrayList** kullanırız. **ArrayList**, .Net için standart olarak tanımlı **IList** **ara yüzünü** (**interface**) uygulayan bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
using System.Collections;

ArrayList liste = new ArrayList();
liste.Add("Renault");
liste.Add("Fiat");
liste.Add("Toyota");
Console.WriteLine("Araçlar");
Console.WriteLine("    Adet:    {0}", liste.Count); //3
Console.WriteLine("    Kapasite: {0}", liste.Capacity); //4
Console.WriteLine("    Değerler:");
foreach (Object obj in liste)
    Console.WriteLine("    {0}", obj); //    Renault    Fiat    Toyota
```

## Jenerik

C# dili farklı veri tipleriyle çalışan yöntemler, sınıflar, algoritmaları tekrar tekrar yazmamak için bir programlama türü olan **jenerik programlamayı** (**generic programming**) destekler. Dizi dışındaki tüm koleksiyonlar jenerik programa ile çalışır.

**Jenerikler** (**generic**), veri tiplerine bağlı olmadan algoritma içeren bir yöntem veya sınıf oluşturmamızı sağlar! Jenerik, aynı mantığı yeniden yazmadan bir yöntemin farklı veri tipleri üzerinde çalışmasını sağlayan bir sınıf, yöntem ve algoritma planı tanımlar. Jenerikler, kodunuzda sınıfı veya yöntemi kullanana kadar bir veya daha fazla tip parametresinin belirtilmesini erteleyen sınıflar ve yöntemler tasarlamayı mümkün kılar.

Genel bir veri tipini temsil eden **T** veya **V** parametresi kullanarak, çalışma zamanı dönüştürmelerinin veya **kutulama** (**boxing**) işlemlerinin maliyetine veya riskine katlanmadan diğer istemci kodunun kullanabileceği tek bir sınıf yazabilirsiniz;

```
// See https://aka.ms/new-console-template for more information
GenericList<int> list1 = new(); //int veri tipinde elemanlardan oluşan liste
list1.Add(1);

GenericList<string> list2 = new(); //string veri tipinde elemanlardan oluşan liste
list2.Add("");

GenericList<AClass> list3 = new(); //AClass veri tipinde elemanlardan oluşan liste
list3.Add(new AClass());

public class GenericList<T>
{
    public void Add(T item) { }
}

public class AClass { } // Örnek Bir Sınıf
```

**IEnumerable**, numaralandırılabilen **ArrayList** gibi tüm genel olmayan koleksiyonlar için temel **ara yüzüdür** (**interface**). **IEnumerator<T>** ise **List<T>** gibi tüm genel numaralandırıcılar için temel ara yüzüdür.

`IEnumerable`, `GetEnumerator` metodunu uygulayan bir ara yüzdür. `GetEnumerator` metodu, `foreach` gibi koleksiyonda yineleme yapmak için seçenekler sağlayan bir `IEnumerator` döndürür.

```
// See https://aka.ms/new-console-template for more information
using System.Collections;
foreach (var araç in new AraçKoleksiyonu())
{
    Console.WriteLine($"{araç} "); //sedan heçbek siteyşın limuzin
}

public class AraçTipiNumaralandırma : IEnumerator<string>
{
    string[] araçTipleri = new string[4] { "sedan", "heçbek", "siteyşın", "limuzin" };
    int currentIndex = -1;
    public string Current
    {
        get
        {
            if (currentIndex < 0 || currentIndex >= araçTipleri.Length)
                throw new InvalidOperationException();
            return araçTipleri[currentIndex];
        }
    }
    object IEnumerator.Current // Explicit implementation of IEnumerator.Current
    {
        get
        {
            return Current;
        }
    }
    public void Dispose()
    {
    }
    public bool MoveNext()
    {
        currentIndex++;
        if (currentIndex < araçTipleri.Length)
        {
            return true;
        }
        return false;
    }
    public void Reset()
    {
        currentIndex = 0;
    }
}

public class AraçKoleksiyonu : IEnumerable<string>
{
    private AraçTipiNumaralandırma enumerator;
    public AraçKoleksiyonu()
    {
        enumerator = new AraçTipiNumaralandırma();
    }
    public IEnumerator<string> GetEnumerator()
    {
        return enumerator;
    }
    IEnumerator IEnumerable.GetEnumerator()
    // Explicit implementation of IEnumerable.GetEnumerator
    {
        return GetEnumerator();
    }
}
```

```
}
}
```

## yield Anahtar Kelimesi

Bir ifadede **yield** anahtar kelimesini kullandığınızda, içinde görüldüğü yöntem, operatör veya **get** erişimcisinin bir **yineleyici** (**iterator**) olduğunu belirtirsiniz. Bir yineleyiciyi tanımlamak için **yield** kullanmak, özel bir koleksiyon türü için **IEnumerable** ve **IEnumerator** desenini uyguladığınızda açık bir ek sınıfa (bir sayım için durumu tutan sınıf) olan ihtiyacı ortadan kaldırır.

```
// See https://aka.ms/new-console-template for more information
using System.Collections.Generic;
public class Program
{
    public static void Main()
    {
        foreach (int value in Count(start: 4, count: 10))
        {
            Console.Write(value+" ");
        }
    }
    public static IEnumerable<int> Count(int start, int count)
    {
        for (int i = 0; i <= count; i++)
        {
            yield return start + i;
        }
    }
}
/*Program Çıktısı:
4 5 6 7 8 9 10 11 12 13 14
*/
```

Yukarıdaki örnekte **foreach** ifadesi gövdesinin her **yinelemesi** (**iteration**), **Count** yineleyici yöntemine bir çağrı oluşturur. Yineleyici yöntemine yapılan her çağrı, for döngüsünün bir sonraki yinelemesi sırasında gerçekleşen **yield return** ifadesinin bir sonraki icrasına ilerler.

Aşağıda bağlı listelere örnek olarak örnek bir **GenericList** jeneriği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
public class GenericList<T>
{
    private class Node(T t) // T tipinde Düğüm verileri içeren iç sınıf
    {
        public T Data { get; set; } = t; // T tipinde özellik (property)
        public Node? Next { get; set; }
    }
    private Node? head; // Bağlı Listede İlk Düğüm
    public void AddHead(T t)
    {
        Node n = new(t);
        n.Next = head;
        head = n;
    }
    public IEnumerator<T> GetEnumerator()
    {
        Node? current = head;
        while (current is not null)
        {
            yield return current.Data;
            current = current.Next;
        }
    }
}
```

```
}
public class Client
{
    public static void Main()
    {
        GenericList<int> list = new();
        for (int x = 0; x < 10; x++)
        {
            list.AddHead(x);
        }
        foreach (int i in list)
        {
            Console.WriteLine(i);
        }
    }
}
```

Mevcut yineleyici yöntemlerinin işlevselliğinde bir sonlandırma koşulu tanımlanırsa iç döngünün yürütülmesini durdurmak için bir **yield break** çağırılır;

```
public static IEnumerable<int> CountUntilAny(int start, HashSet<int> earlyTerminationSet)
{
    int curr = start;
    while (true)
    {
        if (earlyTerminationSet.Contains(curr))
        {
            yield break; // Yinelemeyi durduracak bir değerle karşılaştık
        }
        yield return curr;
        if (curr == Int32.MaxValue)
        {
            yield break; // taşma olmasın diye yinelemeyi durdurduk.
        }
        curr++;
    }
}
```

## HashSet<T> Karma Kümesi

**Karma kümesi** (**hashset**), üyeleri tekil olan bir jenerik koleksiyondur. Dizilerden farklı olarak karma küme içinde bir değer yalnızca bir defa yer alabilir.

```
// See https://aka.ms/new-console-template for more information
HashSet<int> aidatVerenDaireler = new HashSet<int>();
aidatVerenDaireler.Add(4);
aidatVerenDaireler.Add(6);
aidatVerenDaireler.Add(9);
aidatVerenDaireler.Add(10);
Console.WriteLine($"Aidat Veren Daire Sayısı:{aidatVerenDaireler.Count}");
//Karma Küme Başlatma (İlk Değer Verme)
HashSet<int> aidatVermeyenDaireler = new HashSet<int>() { 1, 2, 3, 5, 8 };
bool vermemişmi = aidatVermeyenDaireler.Contains(8);
Console.WriteLine($"8 Numaralı Daire Aidat Vermemiş Mi:{vermemişmi}");
```

## Dictionary<TKey, TValue> Sözlük

**Sözlük** (**dictionary**) koleksiyonu anahtar-değer ikilisi şeklinde sözlük nesnesi oluşturmak için kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
var kişiler = new Dictionary<int, string> { { 10, "İlhan" },
```

```

        { 20, "Mehmet" },
        {30, "Abdullah"} };

Console.WriteLine(kişiler[10]); // "İlhan"
//Console.WriteLine(kişiler[50]); // HATA!:throws KeyNotFoundException
string? adı; //null olabilen string
if (kişiler.TryGetValue(20, out adı))
{
    Console.WriteLine(adı); // "Mehmet"
}
kişiler[40]="Abdullah";
kişiler.Add(50,"Damla");
//kişiler.Add(40, "Damla"); // HATA!:Throws ArgumentException since 40 already exists
foreach (KeyValuePair<int,string> kişi in kişiler)
{
    Console.WriteLine("No={0}, Adı={1}", kişi.Key, kişi.Value);
}
foreach (int no in kişiler.Keys)
{
    Console.WriteLine("No={0}", no);
}
foreach (string adı in kişiler.Values)
{
    Console.WriteLine("Adı={0}", adı);
}

```

## SortedSet<T> Sıralı Küme

**Sıralı küme** (**sortedset**), kümenin elemanlarını sıralı bir şekilde saklamak için kullanılan jenerik bir koleksiyondur. Aynı eleman kümede ikinci defa yer almaz!

```

// See https://aka.ms/new-console-template for more information

var sıralıListe = new SortedSet<int>();
sıralıListe.Add(4);
sıralıListe.Add(3);
sıralıListe.Add(3);// Kümede var eklenmez!
sıralıListe.Add(2);
sıralıListe.Add(1);
foreach (var item in sıralıListe)
{
    Console.Write(item+" ");
}
/*Program Çıktısı:
1 2 3 4
*/

```

## List<T> Liste

Öğeleri bir **liste** (**list**) olarak tutar ve öğeler indekse göre eklenebilir, yerleştirilebilir, kaldırılabilir ve adreslenebilir. Listeler bu amaçla kullanılan jenerik bir koleksiyondur.

```

// See https://aka.ms/new-console-template for more information
var liste = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
liste.Add(6);
Console.WriteLine(liste.Count); // 6
liste.RemoveAt(3);
Console.WriteLine(liste.Count); // 5
Console.WriteLine(liste[3]); // 5

```

## Stack<T> Yığın

**Yığın** (*stack*), koleksiyona son giren ilk geri alınacak şekilde dinamik bir veri yapısı sunan ve bu amaçla kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
var yığın = new Stack<int>();
yığın.Push(10);
yığın.Push(3);
yığın.Push(5);
yığın.Push(8);
Console.WriteLine(yığın.Peek()); // 8
Console.WriteLine(yığın.Pop()); // 8
Console.WriteLine(yığın.Pop()); // 5
Console.WriteLine(yığın.Pop()); // 3
```

## LinkedList<T> Bağlı Liste

Bu koleksiyon **bağlı liste** (*linkedlist*) dinamik veri yapısını sunan ve bu amaçla kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
LinkedList<int> bağlıListe = new LinkedList<int>();
bağlıListe.AddLast(3);
bağlıListe.AddLast(5);
bağlıListe.AddLast(8);
bağlıListe.AddFirst(10);
bağlıListe.AddFirst(2);
bağlıListe.RemoveFirst();
bağlıListe.RemoveLast();
foreach (int i in bağlıListe)
    Console.Write(i+" ");
/* Program Çıktısı:
10 3 5
*/
```

## Queue<T> Kuyruk

Bu koleksiyon **kuyruk** (*queue*) dinamik veri yapısını sunan ve kullanılan jenerik bir koleksiyondur.

```
// See https://aka.ms/new-console-template for more information
var kuyruk = new Queue<int>();
kuyruk.Enqueue(10);
kuyruk.Enqueue(6);
kuyruk.Enqueue(4);
kuyruk.Enqueue(9);
Console.WriteLine(kuyruk.Peek()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 10
Console.WriteLine(kuyruk.Dequeue()); // 6
Console.WriteLine(kuyruk.Dequeue()); // 4
```

## İndisleyiciler

**İndisleyiciler** (*indexer*), nesneleri bir dizi gibi kullanmayı sağlarlar. İndisleyiciler, köşeli parantez içinde parametresi olan **this** kelimesiyle tanımlanırlar. Özellikler gibi indisleyiciler de **get** ve **set** yöntemleri kullanılır ve yalnız-okunur, yalnız-yazılır olabilirler. Bunların dışında indisleyiciler sanal olarak da tanımlanabilir ve parametresi farklı olmak kaydıyla indisleyicilere **aşırı yükleme** (*overload*) yapılabilir.

```
// See https://aka.ms/new-console-template for more information
List<string> names = new List<string>();
names.Add("Ali");
names.Add("Veli");
```

```
names.Add("İlhan");
for (int i = 0; i < names.Count; i++)
{
    //İndis kullanımı:
    string s = names[i];
    names[i] = s.ToUpper();
    Console.WriteLine(names[i]);
}

public class List<T>
{
    //Sabitler:
    const int defaultCapacity = 4;
    //Alanlar:
    T[] items;
    int count;
    //Yapıcılar:
    public List() : this(defaultCapacity) { }
    public List(int capacity)
    {
        items = new T[capacity];
    }
    //Özellikler:
    public int Count
    {
        get { return count; }
    }
    public int Capacity
    {
        Get { return items.Length; }
        set
        {
            if (value < count) value = count;
            if (value != items.Length)
            {
                T[] newItems = new T[value];
                Array.Copy(items, 0, newItems, 0, count);
                items = newItems;
            }
        }
    }
}

//İndisleyici Tanımı:
public T this[int index]
{
    get { return items[index]; }
    set { items[index] = value; }
}

//Yöntemler:
public void Add(T item)
{
    if (count == Capacity) Capacity = count * 2;
    items[count] = item;
    count++;
}
}
```